

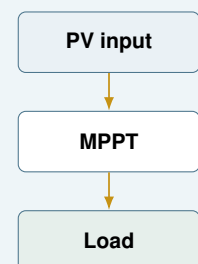


MPPT Circuit Student Implementation Guide

Module ENR 420

Photovoltaic maximum power point tracking for safe laboratory implementation

A practical guide to the MPPT circuit, sensing interfaces, gate-driver control, STM32 pin mapping, firmware architecture, and safe laboratory bring-up used in Practicals 1 and 3.



Contents

1	How This Guide Should Be Used	1
1.1	Assumed background	1
2	System Overview	2
3	Circuit Description	3
3.1	Four-switch buck-boost converter	3
3.2	Ratings and component values	3
4	Safety and Operating Envelope	5
4.1	Panel operating expectations	5
4.2	Firmware fault limits	5
5	PCB and External Sensor Wiring	6
5.1	Four temperature probes at J8	7
5.1.1	Identify and label Temp0–Temp3	7
5.2	Irradiance sensor at J6	8
6	Software Setup	9
6.1	Which STM32 applications are required?	9
6.2	Download and import the firmware	9
6.3	Build, flash, and start execution	9
6.4	Install and run the monitoring GUI	9
7	Supplied STM32 Configuration	10
8	Voltage and Current Sensing	11
8.1	ADC conversion	11
8.2	Voltage sensors	11
8.3	Current sensors	12
8.4	Power calculation	12
9	Irradiance Sensing and RS-485 Operation	13
9.1	The Physical Layer: RS-485 Direction Control	13
9.2	The Protocol Layer: Modbus-RTU	13
10	Firmware Operation	14
10.1	Main control-loop timing	14
10.2	State machine	14
10.3	Supplied P&O MPPT controller	14
10.4	PC console commands	15
10.5	Telemetry and four temperature channels	15
10.6	Fault handling	15
11	Troubleshooting	16
12	Reference Resources	17

1 How This Guide Should Be Used

Welcome to your final-year power electronics project! If your previous experience was writing a basic PID controller on a microcontroller, this project is a fantastic step up. You are going to build the brain for a custom power electronics converter to extract the maximum possible power from solar panels.

This guide explains the system architecture, equations, hardware, wiring, and supplied firmware for the [STM32 NUCLEO-F303K8](#). The package already includes peripheral setup, measurement averaging, a P&O MPPT controller, sensor protocols, telemetry, and a state machine. Your engineering work is to understand that implementation, validate it on the actual hardware, calibrate the measurements, tune it for the practical, and select approved operating limits.

Learning outcomes

After working through this guide you should be able to:

- explain how the four-switch buck-boost converter changes the solar-panel operating point;
- identify each PCB signal connected to the STM32 and verify the supplied GPIO, ADC, UART, and timer configuration;
- convert raw ADC counts into real-world voltage, current, power, and irradiance values;
- explain and test the supplied state machine, measurement checks, duty limits, and fault handling;
- communicate with digital sensors and your PC over serial protocols;
- verify the firmware in the laboratory before connecting the full solar-panel and battery system.

1.1 Assumed background

You are expected to know the basic operation of DC-DC converters, MOSFET switching, PWM, ADC sampling, UART communication, and C programming. The guide fills in the project-specific details: which signals exist on this PCB, how the sensor outputs relate to physical units, and how the supplied STM32 firmware is organised.

Safe laboratory sequence

Do not connect the solar panels, battery, or load during first firmware testing. First confirm that the STM32 boots, the gate-driver disable pins keep switching disabled, ADC readings are plausible, and PWM waveforms are correct on an oscilloscope. Only then proceed to current-limited converter testing.

2 System Overview

Solar panels are highly non-linear; their power output changes depending on how much current you draw from them. If you draw too little, voltage is high but current is near zero (no power). If you draw too much, voltage collapses to zero (no power).

Your goal is to find the "sweet spot" in the middle: the **Maximum Power Point (MPPT)**.

Because you cannot control the sun, you control the **duty cycle** of a DC-DC converter. By changing the duty cycle, you change the "electrical resistance" the solar panel sees, which shifts its operating voltage and current. The MPPT algorithm observes the resulting panel voltage and current, estimates $P_{PV} = V_{PV}I_{in}$, and adjusts the converter command until the operating point is close to the peak.

Figure 1 shows the complete system with the PV input, measurement feedback, MPPT controller, PWM/gate-driver path, and four-switch buck-boost power stage. The MPPT PCB draws power from two series-connected [Yingli YLM-J 3.0 PRO solar panels](#) and delivers power to the output terminals.

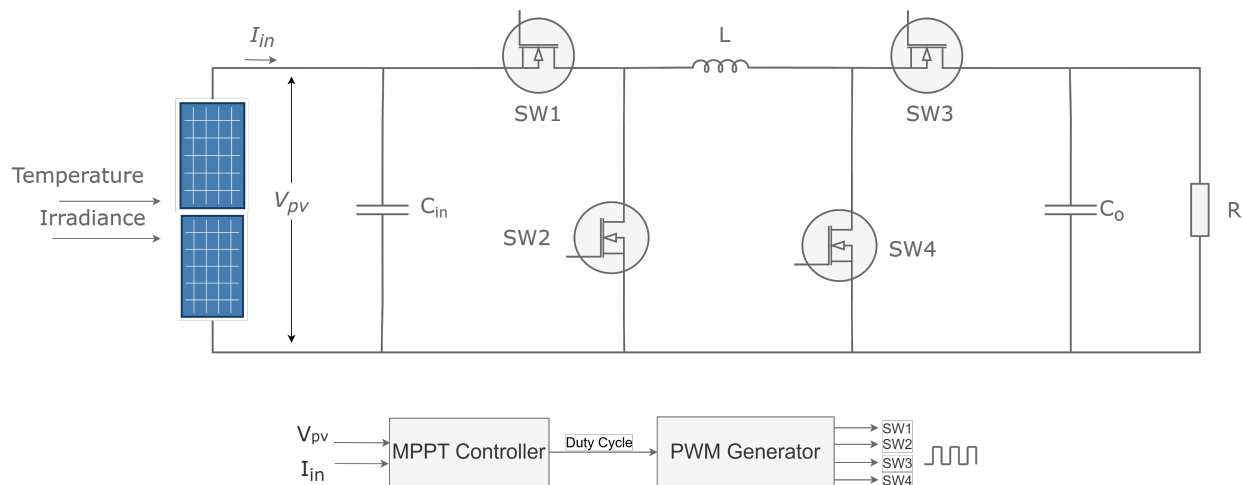


Figure 1: E&R MPPT system diagram

Control objective

For Practical 3, the supplied controller searches for the PV operating point that gives the highest measured input power, $P_{PV} = V_{PV}I_{in}$. The project team must validate the measurement scaling, PWM behaviour, and approved practical operating limits before relying on it with an energised converter.

3 Circuit Description

3.1 Four-switch buck-boost converter

The DC-DC converter is a four-switch buck-boost (4SBB) converter. Because solar voltage fluctuates depending on irradiance and temperature (and your battery/load voltage varies), the 4SBB allows you to step the input voltage down or up without inverting the output polarity.

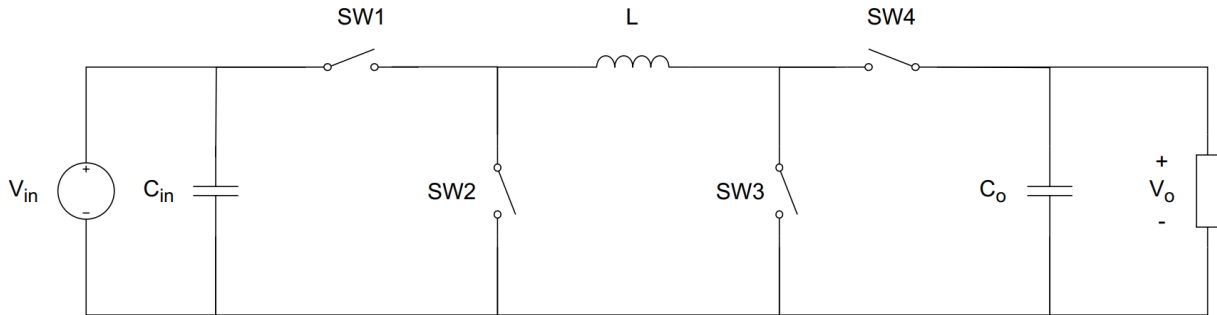


Figure 2: Four-switch buck-boost converter circuit diagram

SW1 and SW2 form the buck half bridge. SW3 and SW4 form the boost half bridge. The inductor transfers energy between the input side and the output side. The input and output capacitors reduce switching ripple and provide local energy storage.

Switch 1 and switch 2 operate with complementary duty cycles of D_1 and $1 - D_1$. Switch 3 and switch 4 operate with duty cycles of D_2 and $1 - D_2$. The ideal continuous-conduction voltage gain is

$$\frac{V_o}{V_{in}} = \frac{D_1}{1 - D_2}. \quad (1)$$

The useful operating cases are:

- **Buck mode:** set $D_2 = 0$. SW4 remains on, SW3 remains off, and the gain is approximately D_1 .
- **Boost mode:** set $D_1 = 1$. SW1 remains on, SW2 remains off, and the gain is approximately $1/(1 - D_2)$.
- **Buck-boost mode:** set both sides to switch, for example $D_1 = D_2 = D$, giving a gain of approximately $D/(1 - D)$.

The supplied controller limits its duty commands and uses a controlled startup. These code-level measures still require oscilloscope verification on the real power stage, and an approved fault must stop switching immediately.

3.2 Ratings and component values

The maximum ratings of the circuit are given in Table 1. These are hardware limits, not firmware targets. The controller must operate below these values and should include a safety margin.

Table 1: Circuit maximum ratings

Parameter	Value
Input voltage	100 V
Input current	15 A
Output voltage	60 V
Output current	25 A

The component values used for the inductor and capacitors are given in Table 2. The circuit is designed for a switching frequency of 250 kHz.

Table 2: Component values

Parameter	Value
C_{in}	25 μ F
L	20 μ H
C_o	15 μ F

At 250 kHz, one PWM period is

$$T_s = \frac{1}{250\,000} = 4\,\mu\text{s}. \quad (2)$$

The commanded duty cycles therefore change the relative on-time of MOSFETs within a $4\,\mu\text{s}$ switching period.

4 Safety and Operating Envelope

The practical system can involve high DC voltage, high current, large capacitors, a battery, and outdoor PV panels. Firmware errors can therefore damage hardware. The first goal of the firmware is safe control. MPPT performance comes after safe startup, measurement sanity checks, and fault handling.

4.1 Panel operating expectations

Two series-connected YLM-J 3.0 PRO modules can produce an open-circuit voltage in the region of twice the single-module open-circuit voltage. A typical YLM-J 3.0 PRO 390–415 W module has V_{MPP} around 30–31 V, I_{MPP} around 13 A, and V_{OC} around 37 V at standard test conditions. Two modules in series therefore place the expected maximum-power voltage near 60–62 V and open-circuit voltage near 74 V before temperature corrections. This is below the PCB input-voltage rating, but it is still hazardous and must be treated as a high-energy DC source.

4.2 Firmware fault limits

Use conservative limits in early tests. The values in Table 3 are suggested starting limits for firmware development and should be confirmed by the demonstrator for the final practical setup.

Table 3: Suggested firmware fault limits for bring-up

Quantity	Initial action limit	Firmware action
Input voltage	Below hardware rating with margin	Disable PWM if the measured value exceeds the approved practical limit.
Input current	Current-limited during first tests	Disable PWM and log a fault if the input current rises unexpectedly.
Output voltage	Below battery/load rating	Disable PWM if output voltage is too high or inconsistent with the load.
Output current	Below practical limit	Disable PWM on overcurrent or negative current if not expected by the test.
Duty cycle	Keep away from 0% and 100%	Clamp commands and ramp slowly.
Sensor validity	ADC and UART values plausible	Do not run MPPT from invalid measurements.

These are not implemented engineering-unit thresholds

Table 3 is a bring-up checklist for limits that must be approved and implemented for the actual laboratory setup. The supplied fault logic detects ADC acquisition failures and saturated ADC values, but it does not claim practical over-voltage, over-current, over-temperature, or communication-loss protection in engineering units.

5 PCB and External Sensor Wiring

A printed circuit board (PCB) has been designed and manufactured for the MPPT circuit. It includes the converter, voltage sensors, current sensors, auxiliary power supplies, gate drivers, microcontroller headers, temperature connector, and irradiance-sensor interface. The unpopulated PCB is shown in Figures 3 and 4.

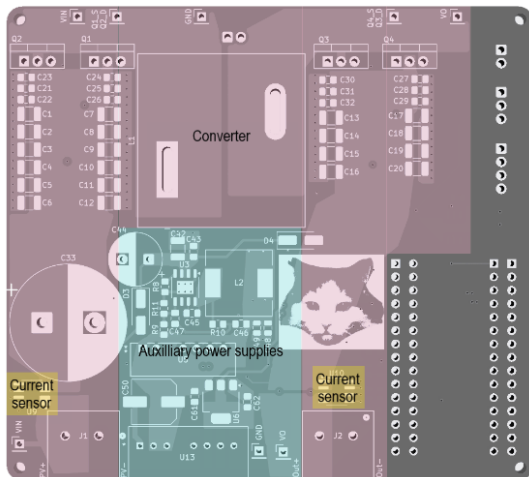


Figure 3: Unpopulated PCB top view

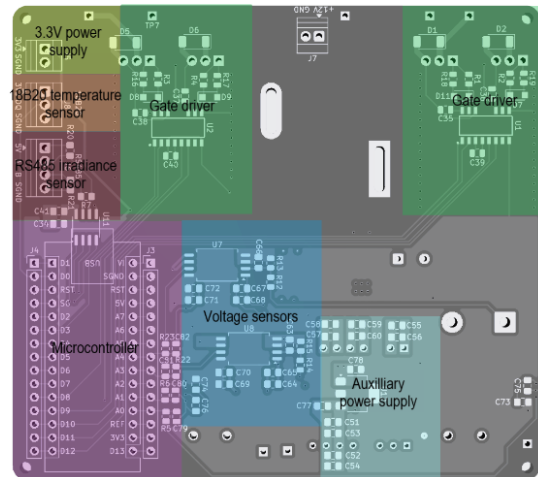


Figure 4: Unpopulated PCB bottom view

The PCB is pictured with components in Figures 5 and 6.

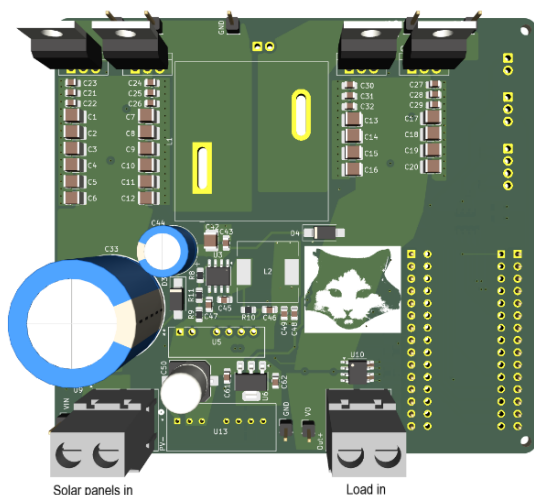


Figure 5: PCB top view with components

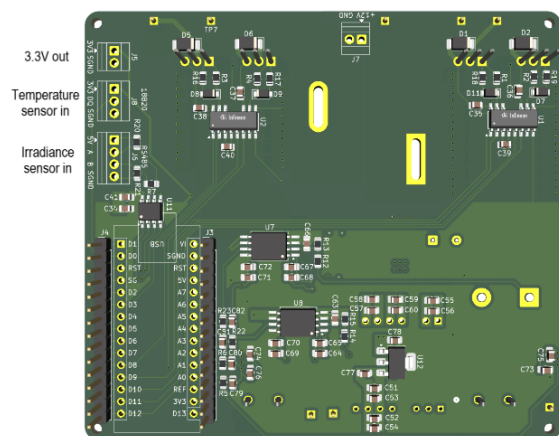


Figure 6: PCB bottom view with components

The high-power PV input and output/load connections are made only according to the practical instructions. Before making those connections, verify polarity with a multimeter and confirm that capacitors are discharged.

Keep sensor wiring separate from the power terminals

J8 and J6 are low-voltage sensor connectors. They are not solar-panel inputs. Never connect a temperature or irradiance lead to the screw terminal labelled “Solar panels in” or to an output/load terminal. Make every sensor connection with all supplies switched off.

5.1 Four temperature probes at J8

The four supplied **DS18B20 probes** share the three-position terminal J8. Join matching conductors in parallel using the approved terminal or distribution harness: all red conductors go to 3V3, all yellow conductors go to DQ, and all black conductors go to SGND. Do not use parasite-power wiring.

Table 4: DS18B20 probe connections to PCB terminal J8

Wire colour	Probe signal	J8 label	Connection
Red	VDD	3V3	3.3 V supply for every probe.
Yellow	Data	DQ	Shared bidirectional 1-Wire data line to PA10 (D0).
Black	Ground	SGND	Shared sensor ground and PCB signal reference.

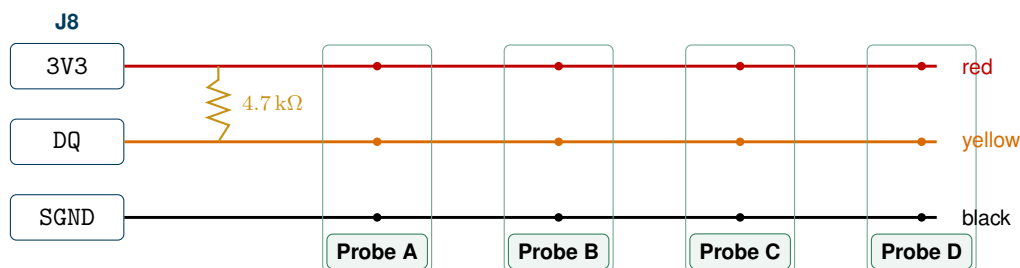


Figure 7: Four DS18B20 probes connected in parallel to the single J8 1-Wire bus

One pull-up resistor for the complete bus

The open-drain DQ bus requires exactly one 4.7 kΩ pull-up to 3V3. First inspect R20 and verify that it is populated with a suitable value between DQ and 3V3. If R20 provides the correct pull-up, do not add another resistor. If R20 is absent or unsuitable, fit one external 4.7 kΩ resistor across J8 DQ and 3V3; it is not placed in series with a wire. Multiple parallel pull-ups reduce the effective resistance and should be avoided.

5.1.1 Identify and label Temp0–Temp3

The firmware discovers devices by their 1-Wire ROM codes. Therefore Temp0 through Temp3 are discovery indices, not fixed PCB sockets or guaranteed panel locations. After every probe set is connected:

1. leave all probe tips together until the four GUI traces are stable;
2. warm one probe tip gently in your hand while watching Temp0–Temp3;

3. label that physical probe with the index whose trace rises, and record its intended panel location;
4. allow it to return towards ambient temperature, then repeat for each remaining probe;
5. repeat the identification if a probe is replaced or the connected set changes.

5.2 Irradiance sensor at J6

The [official SEN0640 pinout](#) identifies the four cable colours shown in Table 5. Connect this sensor to J6, whose PCB labels are 5V, A, B, and SGND.

Table 5: SEN0640 connections to PCB terminal J6

Wire colour	Sensor signal	J6 label	Connection
Brown	VCC	5V	Sensor supply input.
Yellow	485-A	A	RS-485 differential line A.
Blue	485-B	B	RS-485 differential line B.
Black	GND	SGND	Sensor ground and RS-485 signal reference.

Check J6 before inserting the conductors

With the sensor disconnected, confirm the connector labels and approximately 5 V between J6 5V and SGND. Do not connect the brown lead to J8 3V3. If communication fails, re-check A/B polarity without moving either power conductor.

6 Software Setup

6.1 Which STM32 applications are required?

- **Required:** use [STM32CubeIDE](#) to edit, build, flash, and debug the supplied firmware.
- **Not required for this package:** standalone [STM32CubeMX](#) configures peripherals and generates code. Use it when creating a new STM32 project from scratch; the supplied project is already configured.

Do not regenerate this package from Code.ioc

The supplied `Code.ioc` is currently out of sync with the generated ADC, GPIO, timer, and DMA source used by the project. Regenerating from it can overwrite working configuration. Treat the current generated C files as the configuration authority and build them as supplied.

6.2 Download and import the firmware

1. Open the [ENR420 MPPT repository](#) on GitHub.
2. Select **Code**, then **Download ZIP**. Save the archive and extract it to a normal working folder.
3. Start STM32CubeIDE and select a workspace.
4. Select **File** → **Import** → **General** → **Existing Projects into Workspace**, then select **Next**.
5. For the root directory, browse to the extracted repository's top-level `MPPT_firmware` folder. Do not select the nested `MPPT_firmware/STM32CubeIDE` folder.
6. Select the detected project named `Code` and choose **Finish**.

6.3 Build, flash, and start execution

1. Keep the PV input, battery/load, and gate-driver power stage unenergised. Connect the NUCLEO board to the PC through its ST-LINK USB connector.
2. Select the `Code` project and choose **Project** → **Build Project**. Resolve any build error before connecting converter power.
3. Launch the supplied `Code` Debug configuration. STM32CubeIDE builds if necessary, programs the chip through ST-LINK, and opens the Debug perspective.
4. The debugger normally stops at `main()`. Select **Resume** so the firmware runs. The ST-LINK virtual COM port then carries the PC console and telemetry.

6.4 Install and run the monitoring GUI

Open a terminal in the extracted repository root and run:

```
python -m pip install -r requirements-mppt-gui.txt
python mppt_serial_gui.py
```

In the GUI, select the ST-LINK virtual COM port, leave the baud rate at 115200, and select **Connect**. Select **Status** before **Start**; verify that the reported state and fault are appropriate for the test. This COM connection is USART2 and is separate from the SEN0640's USART1 RS-485 link.

7 Supplied STM32 Configuration

The STM32 NUCLEO-F303K8 plugs directly into the PCB headers. Table 6 records the interface implemented by the supplied source; it is a verification map, not a CubeMX task list.

Table 6: STM32 pin and PCB signal map

NUCLEO label	STM32 pin/function	PCB signal label	Purpose
D1	PA9 / TIM1_CH2	BST_LOPWM	PWM signal for the boost-side low switch (SW3).
D3	PB0 / TIM1_CH2N	BST_HIPWM	PWM signal for the boost-side high switch (SW4).
D9	PA8 / TIM1_CH1	BCK_HIPWM	PWM signal for the buck-side high switch (SW1).
D10	PA11 / TIM1_CH1N	BCK_LOPWM	PWM signal for the buck-side low switch (SW2).
D8	PF1 / GPIO output	BST_DIS	Active-high disable signal for the Infineon 2EDB8259Y boost-side gate driver.
D11	PB5 / GPIO output	BCK_DIS	Active-high disable signal for the buck-side gate driver.
D13	PB3 / GPIO output	BST_STP	Shoot-through or dead-time control signal for the boost side.
D12	PB4 / GPIO output	BCK_STP	Shoot-through or dead-time control signal for the buck side.
A0	PA0 / ADC1_IN1	I_IN	ADC feedback for input/PV current.
A1	PA1 / ADC1_IN2	I_0	ADC feedback for output current.
A3	PA4 / ADC2_IN1	V_0	ADC feedback for output voltage.
A4	PA5 / ADC2_IN2	V_IN	ADC feedback for input/PV voltage.
D0	PA10 / open-drain GPIO	DQ	Bidirectional data line for the DS18B20 bus at J8.
D4	PB7 / USART1 RX	R0	Receive data from the ST4E1216IDT RS-485 transceiver .
D5	PB6 / USART1 TX	DI	Transmit data to the ST4E1216IDT.
D6	PB1 / GPIO output	RE	RS485 receiver enable, active low.
D7	PF0 / GPIO output	DE	RS485 driver enable, active high.

Configuration already supplied

The generated source configures the clocks, synchronized two-channel ADC acquisition with DMA, TIM1 complementary PWM and dead time, safe-state GPIO outputs, the open-drain 1-Wire pin, USART1 for the SEN0640, USART2 at 115200 baud for the PC console, and SWD debugging. Read the source to understand these settings; do not reproduce the former CubeMX checklist.

Hardware verification remains essential. With the power stage unenergised, inspect the D1/D3 and D9/D10 complementary pairs on an oscilloscope and confirm the requested approximately 100 ns non-overlap before applying converter power. A successful build does not prove PCB routing or gate-driver polarity, and the stale `Code.ioc` must not be used to correct a discrepancy.

8 Voltage and Current Sensing

The STM32 does not measure panel voltage or current directly. It measures sensor output voltages between approximately 0 and 3.3 V. Firmware must convert ADC counts into engineering units before using them in MPPT or fault checks.

8.1 ADC conversion

For a 12-bit ADC with a reference voltage V_{REF} , the measured ADC input voltage is

$$V_{ADC} = \frac{N}{4095} V_{REF}, \quad (3)$$

where N is the raw ADC count. If the STM32 analog reference is 3.3 V, then one count is approximately 0.806 mV at the ADC input.

ADC readings are inherently noisy in a 250 kHz switching system. The supplied acquisition code uses synchronized DMA sampling and averaging before values reach the MPPT controller. Validate that filtering against oscilloscope and multimeter measurements; adjust it only with evidence that the control timing and transient response remain suitable.

8.2 Voltage sensors

Two TI AMC0311S isolated amplifiers sense input and output voltages. The voltage divider feeds the isolated amplifier input, and the amplifier output is measured by the STM32 ADC. The circuit schematic is shown in Figure 8.

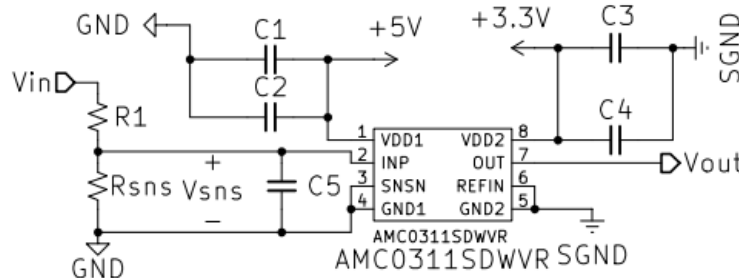


Figure 8: AMC0311S voltage sensor

The input voltage sensor is connected to ADC2_IN2 on A4, and the output voltage sensor is connected to ADC2_IN1 on A3.

Table 7: Voltage sensor parameter values

	Input voltage sensor	Output voltage sensor
R_1 (k Ω)	100	100
R_{sns} (k Ω)	2.2	3.6

Assuming unity gain from the AMC0311S signal path and the divider shown in Figure 8, the voltage

before the divider is estimated from

$$V_{\text{meas}} = V_{\text{ADC}} \left(\frac{R_1 + R_{\text{sns}}}{R_{\text{sns}}} \right). \quad (4)$$

Using the resistor values in Table 7, the approximate scale factors are:

$$V_{\text{in}} \approx V_{\text{ADC}}(46.45), \quad (5)$$

$$V_{\text{o}} \approx V_{\text{ADC}}(28.78). \quad (6)$$

8.3 Current sensors

Two **Allegro CT416** magnetoresistive current sensors measure current. A CT416-HSN820DR unidirectional 20 A sensor is used for the input current, and a CT416-HSN830MR bidirectional ± 30 A sensor is used for the output current. The input current sensor output is connected to ADC1_IN1 on A0. The output current sensor output is connected to ADC1_IN2 on A1.

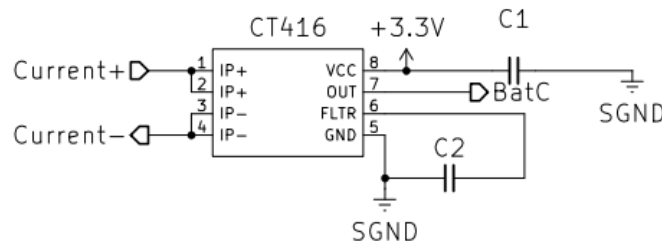


Figure 9: CT416 current sensor

For the CT416-HSN820DR input-current sensor, the nominal sensitivity is 100 mV/A. For a unidirectional sensor, the output is approximately

$$I_{\text{in}} \approx \frac{V_{\text{ADC}} - V_{\text{offset,in}}}{0.100}. \quad (7)$$

The supplied initialization captures the zero-current offset before normal control begins. Verify the captured value with no converter current and repeat the calibration if the sensing hardware or operating conditions change.

For the CT416-HSN830MR output-current sensor, the nominal sensitivity is 33.3 mV/A and the zero-current output is near mid-supply. Estimate output current with

$$I_{\text{o}} \approx \frac{V_{\text{ADC}} - V_{\text{offset,o}}}{0.0333}. \quad (8)$$

8.4 Power calculation

The supplied MPPT controller uses input-side PV power:

$$P_{\text{PV}} = V_{\text{in}} I_{\text{in}}. \quad (9)$$

Output voltage and output current are still important, but they are mainly used for battery/load monitoring, efficiency estimation, and fault detection.

9 Irradiance Sensing and RS-485 Operation

A DFRobot SEN0640 photoelectric solar-radiation sensor is connected at J6 as shown in Section 5. The PCB uses the ST4E1216IDT RS-485 transceiver between the differential bus and STM32 USART1. This sensor link runs independently of the USART2 PC console.

The SEN0640 measures solar radiation over a 400–1100 nm wavelength range and reports values over **Modbus-RTU**. The default Modbus address is 0x01. The default UART settings are 4800 bit/s, 8 data bits, no parity, and 1 stop bit.

9.1 The Physical Layer: RS-485 Direction Control

RS-485 uses differential wires to reject noise and is half-duplex on this board. The firmware controls whether the ST4E1216IDT is driving or listening through Driver Enable (DE) and Receiver Enable (RE):

- To **send** a request to the sensor: Set DE=1.
- To **listen** for the sensor's reply: Set DE=0 and RE=0.

The supplied non-blocking transaction changes from transmit to receive in the USART completion callback, after the last stop bit has left the UART. It then receives the fixed-length reply asynchronously, so the main control loop is not held in a blocking Modbus wait.

9.2 The Protocol Layer: Modbus-RTU

Modbus-RTU defines the request and response byte frames. The SEN0640 is the slave at default address 0x01, and the STM32 is the master. The supplied firmware polls register 0x0000 with function 0x03 and a register count of 0x0001.

For the default address, the request frame is:

Listing 1: SEN0640 read-irradiance request bytes

```
01 03 00 00 00 01 84 0A
```

A response carrying 100 W/m^2 is:

Listing 2: Example SEN0640 response bytes

```
01 03 02 00 64 9B AF
```

Protocol handling already implemented

The firmware builds the request, appends the Modbus CRC-16 in wire order, controls DE/RE, validates the returned address, function, byte count, and CRC, and converts the big-endian 16-bit register to irradiance in W/m^2 . It polls at 5 Hz. Students should verify real sensor responses and A/B polarity, but they do not need to recreate the CRC or byte-order logic.

10 Firmware Operation

The supplied application uses a cooperative state machine and separate drivers for measurement acquisition, the power stage, temperature, irradiance, the PC console, telemetry, and MPPT control. The main loop services these modules without placing blocking sensor or console transactions in the fast control path.

10.1 Main control-loop timing

The current source uses these principal rates and time windows:

- **250 kHz:** TIM1 generates complementary PWM in hardware.
- **1 ms:** the fast task publishes synchronized ADC snapshots and checks acquisition validity.
- **P&O windows:** after a command change, the controller settles for 20 ms and then forms a trimmed mean from eight fresh power samples.
- **50 ms:** one telemetry packet is queued for the PC console (20 packets per second).
- **200 ms:** one SEN0640 Modbus transaction is started (5 polls per second).

USART2 output is placed in an interrupt-driven ring buffer. Telemetry formatting and queuing therefore remain separate from physical UART transmission. If this mechanism is changed, re-check timing and never replace it with an unbounded blocking print in the fast path.

10.2 State machine

The implemented states are shown in Table 8.

Table 8: Implemented firmware states

State	Purpose	Exit condition
INIT	Configure peripherals and force safe outputs.	Initialisation complete.
IDLE	PWM stopped, gate drivers disabled, measurements active.	User/test command and valid measurements.
STARTUP	Apply the conservative initial command and allow a startup settling interval.	Settling interval complete without a detected fault.
RUN	Execute MPPT, update duty cycles, log data.	Fault, stop command, or test complete.
FAULT	Disable switching and record cause.	Manual reset or approved recovery condition.

10.3 Supplied P&O MPPT controller

MpptController.c implements perturb and observe (P&O) using $P_{PV} = V_{in}I_{in}$. It perturbs the requested converter gain, waits for the new operating point to settle, and compares a trimmed-mean power sample with the accepted reference. A clear improvement continues in the same direction; a loss or inconclusive change restores the reference, reverses direction, and returns to the fine step. A power deadband and bounded adaptive step reduce reaction to noise.

The requested gain is mapped to buck duty first and then to boost action at the real power-stage duty limit. `PowerStage_SetDuty()` remains the final clamp before timer compare values are written. The supplied deadband, settling time, and step bounds are initial implementation values; confirm and tune them from controlled tests rather than treating them as universally valid settings.

10.4 PC console commands

USART2 accepts one-character commands through the GUI or another 115200-baud terminal:

Key	GUI control	Action
s	Start	Request entry from IDLE into STARTUP.
x	Stop	Request a controlled return to IDLE.
r	Reset Fault	Clear a latched fault while in FAULT.
?	Status	Print the current state and fault.
h	—	Print the command help.

10.5 Telemetry and four temperature channels

Each new firmware packet contains exactly 19 comma-separated fields in this order:

Listing 3: Telemetry wire format; fields 17 and 18 are appended

```
i_in_raw,i_out_raw,v_out_raw,v_in_raw,valid,
temp0_c,temp1_c,irr_w_m2,state,fault,
mppt_phase,mppt_gain,mppt_step,buck_duty,boost_duty,
reference_power_w,sampled_power_w,temp2_c,temp3_c
```

An unavailable DS18B20 channel is emitted as NA; it is never replaced by zero. The GUI accepts both the earlier 17-field format and the new 19-field format. For CSV recording, the PC receive time `unix_ms` is placed first and `temp2_c`, `temp3_c` are appended last, so every existing CSV column keeps its original position.

10.6 Fault handling

When the supplied code enters FAULT, it:

1. disables the power stage and asserts `BCK_DIS` and `BST_DIS`;
2. stops MPPT updates and latches the fault code;
3. continues the console and telemetry services so the state, fault, and current measurements remain visible; and
4. requires a deliberate reset command before it returns to IDLE.

These actions do not add the missing practical engineering-unit thresholds discussed with Table 3. Confirm the fault cause and make the hardware safe before issuing reset.

11 Troubleshooting

Table 9: Common faults and first checks

Symptom	Likely causes	First checks
No PWM on a pin	Timer not started, wrong alternate function, or gate-driver disable asserted	Check the supplied <code>tim.c/gpio.c</code> , start calls, disable signal, and oscilloscope ground.
Complementary pair overlaps	Dead time missing or wrong channel pairing	Recheck TIM1 complementary-output settings and D1/D3 mapping.
Gate driver does not switch	Disable pin high, supply missing, wrong PWM input	Measure DIS, driver supply, and input pin waveform.
ADC value stuck	Wrong ADC channel, pin not analog, DMA not started	Inspect the generated <code>adc.c</code> rank/DMA setup and raw counts; do not regenerate from <code>Code.ioc</code> .
Current nonzero at no load	Offset capture invalid, noise, or wrong sign convention	Re-run zero-current startup, compare against a meter, and inspect filtering.
Temperature shows NA	Probe absent, J8 miswired, or 1-Wire pull-up missing/incorrect	Check red–3V3, yellow–DQ, black–SGND; then verify R20 or the single external 4.7 k Ω pull-up.
Irradiance timeout	RS-485 direction wrong, A/B reversed, baud mismatch, or bad reply	Check J6 wiring, DE/RE, USART1 settings, and request/response bytes.
MPPT unstable	Settling too short, step too large, noisy scaling, or weak fault limits	Keep the power stage safe, record telemetry, and tune one evidence-backed parameter at a time.
Output voltage rises too high	Load disconnected, duty too high, wrong mode transition	Disable PWM and inspect load, limits, and duty clamp.

12 Reference Resources

Use the following references when understanding the supplied configuration, validating the firmware, or checking component behaviour:

- [STM32CubeIDE download page](#).
- [Standalone STM32CubeMX page](#) for new projects configured from scratch.
- [NUCLEO-F303K8 product page](#).
- [UM1956 STM32 Nucleo-32 boards user manual](#).
- [STM32F303 documentation page](#).
- [STM32F303x6/x8 datasheet](#).
- [RM0316 STM32F303 reference manual](#) for peripheral setup.
- [Infineon 2EDB8259Y gate-driver page](#).
- [TI AMC0311S isolated-amplifier page](#).
- [Allegro CT416 current-sensor datasheet](#).
- [Analog Devices DS18B20 temperature-sensor datasheet](#).
- [ST4E1216 RS485 transceiver datasheet](#).
- [Control Solutions Modbus protocol tutorial](#).
- [DFRobot SEN0640 pinout and product reference](#).
- [DFRobot SEN0640 Modbus-RTU reference](#).
- [Yingli YLM-J 3.0 PRO panel datasheet](#).